12 — File I/O

Dateien lesen und schreiben – das Gedächtnis Ihres Programms

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Wozu Dateien?
2. `open()` – Datei-Handle, Cursor und Puffer
3. Schreiben mit `write()`, die Modi `r` / `w` / `a`
4. Lesen: `read()` und Zeile für Zeile
5. `with` – garantiertes Schließen
6. Text ist nicht gleich Bytes: `encoding`
7. Pfade und Fehlerbehandlung
8. **Klassiker: Wortzähler** – live programmiert

Lernziel: Daten über das Programmende hinaus sichern – und **verstehen**, was zwischen Programm, RAM-Puffer und Festplatte wirklich passiert.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 12).

Wozu Dateien?

Ohne Datei

```
warenkorb = ["Eco-Sneaker", "Hemp-High"]
# Programm endet -> RAM wird freigegeben,
# die Liste ist unwiederbringlich weg
```

Mit Datei – Vorschau auf heute

```
with open("warenkorb.txt", "w") as f:
    f.write("Eco-Sneaker\nHemp-High\n")
```

E4-Rückblick, Speicherhierarchie: RAM ist schnell, aber **flüchtig** – die Platte langsam, aber **dauerhaft**. Bisher lebten alle unsere Daten im RAM. Heute überqueren wir diese Grenze.

`open()` und `close()`

```
f = open("preise.txt", "w")      # 1) oeffnen
f.write("Eco-Sneaker 89.95\n")   # 2) arbeiten
f.close()                        # 3) schliessen
```

- `open(name, modus)` liefert ein **File-Objekt** – eine geöffnete *Verbindung* zur Datei, **nicht** ihren Inhalt
- Zweites Argument: der **Modus** (`"r"` / `"w"` / `"a"`, gleich im Detail)
- `close()` ist Pflicht – **warum**, sehen wir auf den nächsten zwei Folien

Unter der Haube: was `open` wirklich liefert

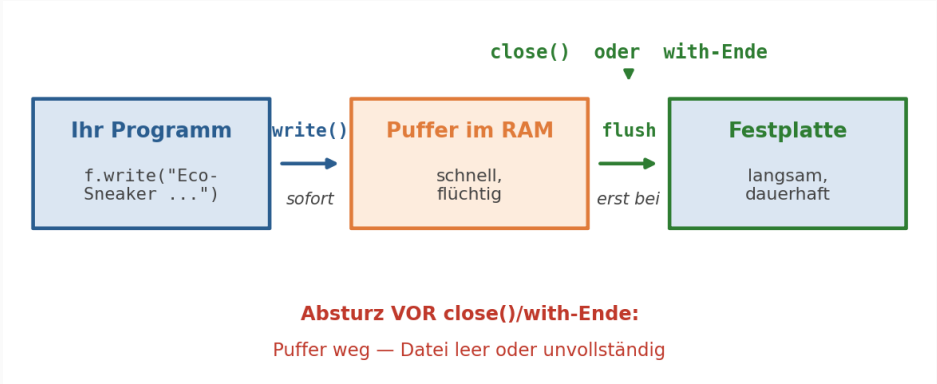
Ein **Handle** mit eingebauter **Leseposition** (Cursor):

```
f = open("bestell_log.txt", "r")
erster = f.read()      # liest ab Cursor: alles
zweiter = f.read()     # Cursor steht am Ende!
f.close()              # -> zweiter == ""
```

Aufruf	Cursor vorher	liefert	Cursor danach
`open(..., "r")`	–	Handle	Position 0
1. `f.read()`	Position 0	ganzer Inhalt	Ende
2. `f.read()`	Ende	`""` (leer!)	Ende

Jedes Lesen bewegt den Cursor weiter – **deshalb** ist die zweite Lesung leer. Nochmal von vorn? Schließen und neu öffnen.

Unter der Haube: der Puffer – warum `close()` Pflicht ist



`write()` schreibt zunächst nur in den **Puffer im RAM** (E4: schnell, flüchtig). Erst `close()` leert den Puffer sicher auf die **Platte** (dauerhaft). Stürzt das Programm vorher ab, ist der Pufferinhalt **weg**.

Schreiben mit `write()`

```
f = open("report.txt", "w")
f.write("Veggie Soles -- Tagesreport\n")
f.write("Eco-Sneaker 89.95 EUR\n")   # \n manuell!
f.close()
```

- `write(text)` schreibt **einen String** – ohne automatischen Zeilenumbruch: `\n` selbst setzen
- Nur Strings: Zahlen vorher per f-String umwandeln

▶ **Live-Demo** – `01_datei_schreiben.py`

✎ **Sofort ausprobieren** – Notebook 12, Angeleitete Übungen 2.1 / 2.2: Text und Einkaufsliste schreiben

Die Modi `r`, `w`, `a` – mit Konsequenzen

Modus	Bedeutung	Datei fehlt	Datei existiert
`"r"`	lesen	**FileNotFoundError**	wird gelesen
`"w"`	schreiben	wird angelegt	**wird geleert!**
`"a"`	anhängen	wird angelegt	wird verlängert

```
open("bestell_log.txt", "w")   # neuer Tag: frisch
open("bestell_log.txt", "a")   # Bestellung anhaengen
open("bestell_log.txt", "r")   # Log auswerten
```

Merkhilfe: **r**ead / **w**rite / **a**ppend. Der gefährliche ist `"w"` – nächste Folie.

Unter der Haube: die `w`-Falle

```
# bestell_log.txt enthaelt 3 Bestellzeilen ...
f = open("bestell_log.txt", "w")   # <- HIER!
f.write("neu\n")
f.close()
```

Zeitpunkt	Inhalt `bestell_log.txt`
vorher	3 Bestellzeilen
nach `open(..., "w")`	**leer** (0 Byte)
nach `write("neu\n")`	nur noch `neu`

`"w"` leert die Datei **beim Öffnen** – kommentarlos, noch **vor** dem ersten `write()`. Bestand erhalten? Dann `"a"`. Es gibt kein Rückgängig.

Lesen: `read()` und Verwandte

```
f = open("bestell_log.txt", "r")
inhalt = f.read()     # EIN String, komplett
f.close()
```

Methode	Liefert
`f.read()`	gesamten Inhalt als String
`f.readline()`	nächste Zeile (mit \n)
`f.readlines()`	Liste aller Zeilen

Faustregel: kleine Dateien → `read()`. Große Dateien → Zeile für Zeile (nächste Folie).

Zeile für Zeile mit `for`

```
f = open("bestell_log.txt", "r")
for zeile in f:                # FS 10: for!
    print(zeile.rstrip("\n"))
f.close()
```

- Das File-Objekt ist **iterierbar**: pro Durchlauf wandert der **Cursor** genau eine Zeile weiter
- Im RAM liegt nur **eine** Zeile – klappt auch bei riesigen Dateien; `rstrip("\n")` entfernt den Umbruch

▶ **Live-Demo** – `02_datei_lesen.py`

✎ **Sofort ausprobieren** – Notebook 12, Übungen 3.1 / 3.2: lesen, Zeilen zählen

Der `with`-Block

```python
with open("report.txt", "w") as f:
    f.write("Eco-Sneaker 89.95 EUR\n")
# hier: garantiert geschlossen & gespeichert
```

- `with` öffnet die Datei und schließt sie **automatisch** am Blockende – `close()` entfällt
- `as f` bindet das Handle an einen Namen
- Gilt auch, wenn im Block ein **Fehler** auftritt – warum, zeigt die nächste Folie

Standard-Muster: in echtem Code *immer* `with`. `open` + `close` war nur der Lehr-Schritt.

Unter der Haube: `with` = das eingebaute `finally`

Was Sie mit Notebook 11 selbst bauen würden

```
f = open("report.txt", "w")
try:
    f.write("Eco-Sneaker 89.95 EUR\n")
finally:
    f.close()      # laeuft IMMER (NB 11!)
```

`with` macht exakt das – in zwei Zeilen

```
with open("report.txt", "w") as f:
    f.write("Eco-Sneaker 89.95 EUR\n")
```

▶ **Live-Demo** – `03_with_block.py` (Datei ist selbst nach Exception zu)

Unter der Haube: Text ist nicht gleich Bytes

```
with open("kunden.txt", "w",
          encoding="utf-8") as f:
    f.write("Grüße an Anna Müller\n")
```

Vorgang	Ergebnis
"ü" schreiben (UTF-8)	2 Bytes: C3 BC
lesen mit UTF-8	ü – korrekt
lesen mit Latin-1	Ã¼ – Umlaut-Salat!

Die Platte speichert nur **Bytes** – das `encoding` übersetzt Zeichen ↔ Bytes (NB 05: Unicode, heute eingelöst). Geben Sie `encoding="utf-8"` immer **explizit** an – sonst rät Python je nach Betriebssystem.

Pfade: wo “ist” das Programm?

```
open("bestell_log.txt")          # relativ
open("daten/log.txt")            # relativ, Ordner
open("/Users/anna/log.txt")      # absolut (macOS)
open(r"C:\Anna\log.txt")         # absolut (Windows)
```

- **Relativ**: ab dem **Arbeitsverzeichnis** – dem Ordner, in dem Python *gestartet* wurde (**nicht** automatisch der Ordner der `.py`-Datei!)
- Häufigste Ursache für “Datei nicht gefunden”: falsches Arbeitsverzeichnis
- **Absolut** = kompletter Weg ab Wurzel; Windows als Raw-String `r"..."`

Faustregel: im Kurs immer relative Pfade – die Datei landet dort, wo Sie das Skript starten.

Fehlerbehandlung beim File I/O

```
try:
    with open("preise.txt", "r") as f:
        text = f.read()
except FileNotFoundError:
    print("Nicht da -- starte mit leerem Log.")
    text = ""
```

Exception	Wann?
`FileNotFoundError`	`"r"`, aber Datei fehlt
`PermissionError`	Betriebssystem verweigert Zugriff
`UnicodeDecodeError`	Encoding passt nicht (eben gesehen!)

▶ **Live-Demo** – `04_fehler_pfade.py`

Klassiker: Wortzähler – die Aufgabe

Der Unix-Klassiker `wc` (*word count*, seit 1971) – heute in Python, für unser Bestell-Logbuch:

- Erzeuge **zuerst selbst** die Datei `bestell_log.txt` mit 4 Bestellzeilen (je: Kunde bestellt Produkt)
- Lies sie zeilenweise und zähle **Zeilen** und **Wörter** (`split()`)
- Schreibe das Ergebnis in `report.txt` **und** auf die Konsole

So soll es laufen

```
bestell_log.txt: 4 Zeilen, 16 Woerter
Report gespeichert: report.txt
```

Klassiker: Wortzähler – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_wortzaehler.py`
Teilziele:

1. Logdatei mit `with` + `"w"` erzeugen (4 Bestellzeilen)
2. Mit `with` + `"r"` zeilenweise lesen (`for zeile in f:`)
3. Zwei Zähler-Akkumulatoren `zeilen` / `woerter` (FS 10!)
4. Pro Zeile: `zeile.split()` → Wortliste, `len()` addieren
5. Report mit `"w"` schreiben + Konsolenausgabe

✎ **Zum Selberlösen** – Handout *“Klassiker 12: Wortzähler”* (`Klassiker/12_Klassiker_Wortzaehler.pdf`). Vertiefung: Notebook 12, Abschlussübungen 3 und 4.

Cheat Card

Aufgabe	Syntax
Sicher öffnen	`with open(name, modus) as f:`
Modi	`"r"` lesen, `"w"` leert!, `"a"` anhängen
Encoding	`open(..., encoding="utf-8")`
Schreiben	`f.write("Text\n")`
Alles lesen	`inhalt = f.read()`
Zeile für Zeile	`for zeile in f:`
Zeile in Wörter	`zeile.split()`
Fehlerschutz	`try: ... except FileNotFoundError:`
Klassiker heute	wc: Log lesen → zählen → Report

Faustregel: `with` + `encoding="utf-8"` + relative Pfade + spezifisches `except` – damit sind 90 Prozent aller Dateijobs sauber gelöst.

Ausblick: Notebook 13a – Algorithmen modellieren

Unser Wortzähler ist ein vollständiger **Algorithmus**: Eingabe (Datei) → Verarbeitung (zählen) → Ausgabe (Report).

Aber heute haben wir einfach **drauflosgetippt**. Zwei Fragen blieben offen:

- Wie **plant** man einen Ablauf, *bevor* man ihn tippt? → **Pseudocode** und **Flussdiagramme**
- Was, wenn das Logbuch **10 Millionen** Zeilen hat – wie *teuer* ist unser Zähler? → **Komplexität**

Nächste Woche: Algorithmen erst **modellieren**, dann implementieren.

Heute geübt

- ✓ Dateien = dauerhafter Speicher (E4: RAM flüchtig, Platte bleibt)
- ✓ `open()` liefert ein **Handle mit Cursor** – zweites `read()` ist leer
- ✓ Puffer im RAM – erst Schließen sichert die Daten auf die Platte
- ✓ Modi `r` / `w` / `a` – und die `w`-Falle (leert beim Öffnen!)
- ✓ `read()`, `for zeile in f:`, `rstrip("\n")`
- ✓ `with` = eingebautes `finally` (NB 11) – Standard-Muster
- ✓ `encoding="utf-8"` (NB 05 eingelöst), Pfade, `FileNotFoundError`
- ✓ **Klassiker Wortzähler** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Wortzähler** eigenständig lösen (Handout, mit Bonus)
- “Sofort ausprobieren”-Aufgaben im Notebook 12
- Abschlussübungen 1–4 im Notebook 12 (mit Musterlösungen)